Project Charter

Manufacturing AI Decision Intelligence Platform

Objective

Design and develop a production-inspired AI-enabled manufacturing platform that ingests operational manufacturing data, processes it using a modern lakehouse architecture, trains predictive machine learning models, and provides engineers with AI-powered decision support through a Retrieval-Augmented Generation (RAG) assistant.

The platform demonstrates the complete lifecycle of modern data systems—from ingestion and engineering to machine learning, deployment, and AI applications.

Primary Goals

Data Engineering

Build scalable batch data pipelines using Databricks and PySpark.

Machine Learning

Predict equipment failures using engineered operational features and track experiments with MLflow.

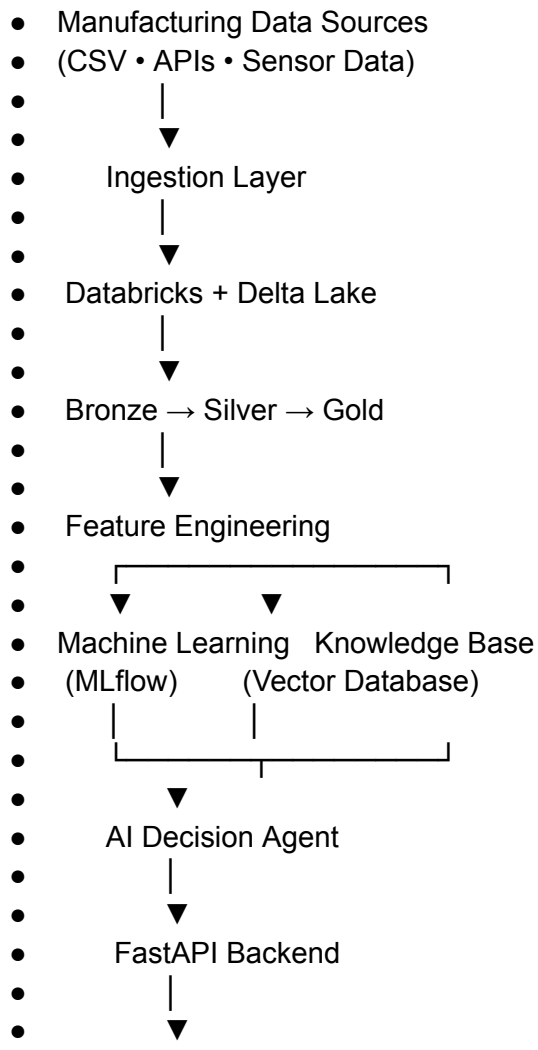
AI

Enable natural language interaction with manufacturing documentation and operational data using a RAG-powered AI agent.

Software Engineering

Deploy the platform as production-style services using FastAPI, Docker, CI/CD, and automated testing.

System Architecture



Repository Structure

- manufacturing-ai-platform/
-
- ├── data/
- │
- ├── ingestion/
- │ ├── api_ingestion.py
- │ ├── csv_ingestion.py
- │ └── validation.py
- │
- ├── spark/
- │ ├── bronze.py
- │ ├── silver.py
- │ └── gold.py
- │
- ├── features/
- │ ├── build_features.py
- │ └── feature_store.py
- │
- ├── analysis/
- │ ├── exploratory_analysis.ipynb
- │ ├── feature_analysis.ipynb
- │ └── model_interpretation.ipynb
- │
- ├── ml/
- │ ├── train.py
- │ ├── evaluate.py
- │ ├── predict.py
- │ └── mlflow/
- │
- ├── rag/
- │ ├── ingest_documents.py
- │ ├── embeddings.py
- │ └── retrieval.py
- │
- ├── vector_db/
- │
- ├── agent/
- │ ├── tools.py
- │ └── agent.py
- │
- ├── api/

- │ └── main.py
- │
- ├── dashboard/
- │
- ├── tests/
- │
- ├── docker/
- │
- ├── .github/
- │ └── workflows/
- │
- ├── docker-compose.yml
- │

└── README.md

Technologies

Data Engineering

- Python
- SQL
- Databricks
- PySpark
- Delta Lake
- Medallion Architecture
- Git

Machine Learning

- Scikit-learn
- XGBoost
- MLflow

Data Science

- Pandas
- NumPy
- Matplotlib
- Feature Engineering
- Statistical Analysis
- Model Explainability (SHAP)

AI

- LangGraph
- OpenAI API (or local model)
- RAG
- ChromaDB (or Qdrant)

Backend

- FastAPI

DevOps

- Docker
- Docker Compose
- GitHub Actions

Dashboard

- Streamlit

Core Features

Data Platform

- Batch ingestion from multiple manufacturing data sources
- Delta Lake Bronze/Silver/Gold architecture
- Data validation and quality checks
- Business-ready Gold datasets

Data Science

- Exploratory Data Analysis (EDA)
- Feature engineering
- Model comparison
- Hyperparameter tuning
- Cross-validation
- Model evaluation
- SHAP explainability

Machine Learning

Predict:

- Equipment failures
- Maintenance priority
- Remaining useful life (optional stretch goal)

Track:

- Experiments
- Metrics
- Registered models
- Model versions

using MLflow.

AI

Provide an AI assistant capable of answering questions such as:

- "Why is Machine A predicted to fail?"
- "Summarize recent maintenance issues."
- "Find similar historical failures."
- "Recommend troubleshooting steps from the maintenance manual."

API

Expose REST endpoints for:

- Predictions
- AI chat
- Equipment metrics
- Health checks

Dashboard

Display:

- Manufacturing KPIs
- Failure predictions
- Equipment health
- AI assistant
- Model performance

Skills Demonstrated

Data Engineering

- ETL / ELT
- Data Modeling
- Distributed Processing
- Lakehouse Architecture
- Data Validation

Data Science

- Exploratory Data Analysis
- Feature Engineering
- Statistical Analysis
- Model Selection
- Model Explainability

Machine Learning

- Predictive Modeling
- Experiment Tracking
- Model Evaluation
- Model Deployment

AI Engineering

- Retrieval-Augmented Generation
- AI Agents
- Vector Search
- LLM Integration

Software Engineering

- REST APIs
- Docker
- CI/CD
- Automated Testing
- Git

Why I think this is the right amount of scope

I **don't** think you should try to build a "mega project" with 25 disconnected technologies. Instead, this project uses around a dozen technologies that naturally fit together because they represent the lifecycle of a real AI-enabled data platform.

The key is to treat this as a phased project:

- **Phase 1:** Build the data platform (Databricks, PySpark, Delta Lake, FastAPI, dashboard).
- **Phase 2:** Add the machine learning workflow (feature engineering, XGBoost, MLflow, SHAP).
- **Phase 3:** Add the AI capabilities (RAG, vector database, LangGraph agent).
- **Phase 4:** Polish it with testing, Docker, GitHub Actions, and documentation.

That approach keeps the project manageable while giving you a portfolio piece that is both technically deep and easy to defend in interviews. Every technology has a clear purpose, and

you can explain how each layer builds on the previous one rather than feeling like it was added just to increase the keyword count.